

DATA-260 Homework 2 Report

Stateful rental-housing agent graph and validation experiments

Student	Pragya Chourasia
SID4	6758
Domain	Rental housing listings
PORT_BASE	8458
PREFIX	s6758
SEED	6758
VERIFY_SEED	266758
Local model	llama3.2:3b
Temperature	0.0
Repository	https://github.com/pragyachourasia-05/data260-6758
Submission tag	hw2

1. Project Overview

Homework 2 extends the shared rental-housing codebase from HW1. The FastAPI application serves the browser interface on port 8458. The graph uses shared typed state, Supervisor, Planner, and Reviewer nodes. Model calls remain routed through the local adapter in `src/model_client.py`.

The Supervisor advances the turn count. The router sends an empty proposal to Planner, a proposal without feedback to Reviewer, and an issue-bearing review back to Planner. A separate validation gate checks the raw Planner response before Reviewer runs, so schema retries and Reviewer correction cycles remain distinguishable.

Reproducible commands

```
python .\main.py
python .\graph_demo.py --model llama3.2:3b --strict
python .\measure_schema_retries.py --runs 30 --model llama3.2:3b --validation_ceiling 3
```

Raw inputs and outputs are stored in `reports/hw02/cases/` and `reports/hw02/raw/`.

2. Part 1 - HTML/CSS

The page was checked at a 375px viewport. The form remains readable and the primary controls fit within the narrow layout. The interface also includes visible loading, empty, and error handling paths in the client code.

localhost:8458

Rental Housing Listings

Property Address *

e.g. 123 Main St, San Jose, CA

Monthly Rent *

e.g. \$2,200/month

Your Email *

you@sjsu.edu

Property Description *

Describe the property: bedrooms, bathrooms, square footage, amenities, pet policy, etc.

Category *

Choose

☐ I agree to the terms and conditions.

Submit Listing

Figure 1. Responsive rental-housing form at a narrow viewport.

3. Part 2 - FastAPI operations

The FastAPI backend supports creating a record, updating record ID 1, deleting the highest-ID record, and searching by the primary or secondary field. Each browser capture below shows the resulting home view after an operation.

localhost:8458

e.g. 123 Main St, San Jose, CA

Monthly Rent *

e.g. \$2,200/month

Your Email *

you@sjsu.edu

Property Description *

Describe the property: bedrooms, bathrooms, square footage, amenities, pet policy, etc.

Category *

Choose

☐ I agree to the terms and conditions.

Submit Listing

Listings

Search by address or rent...

Search

Clear

283 Taft street (#1)

House | 2200

Edit

Delete

e.g. 123 Main St, San Jose, CA

Monthly Rent *

e.g. \$2,200/month

Your Email *

you@sjsu.edu

Property Description *

Describe the property: bedrooms, bathrooms, square footage, amenities, pet policy, etc.

Category *

Choose

☐ I agree to the terms and conditions.

Submit Listing

Listings

Search by address or rent...

Search

Clear

283 Taft street (#1)

House | 5000

Edit

Delete

localhost:8458

you@sjsu.edu

Property Description *

Describe the property: bedrooms, bathrooms, square footage, amenities, pet policy, etc.

Category *

Choose

☐ I agree to the terms and conditions.

Submit Listing

Listings

Search by address or rent...

Search

Clear

283 Taft street (#1)

House | 5000

EditDelete

385 River Oaks Parkway (#2)

Apartment | 3000

EditDelete

e.g. \$2,200/month

Your Email *

you@sjsu.edu

Property Description *

Describe the property: bedrooms, bathrooms, square footage, amenities, pet policy, etc.

Category *

Choose

☐ I agree to the terms and conditions.

Submit Listing

Listings

Search by address or rent...

Search

Clear

283 Taft street (#1)

House | 5000

EditDelete

Figures 4-5. Delete result and newly added record

4. Part 3 - Stateful Agent Graph

AgentState carries the task, proposal, feedback, validation status, and turn count. The normal node functions are deterministic graph workers from the routing perspective: Planner proposes, Reviewer checks, and Supervisor updates state before the router chooses the next edge.

For the correction test, the Reviewer hook injects an issue. The router observes it and sends control to Planner, which produces a valid proposal. This demonstrates the self-correction edge rather than a one-way waterfall.

```
(venv) PS C:\SJSU Assignments\Agentic AI\Assignment_1> python .\graph_demo.py --model llama3.2:3b --strict --force_issue
Running graph on: 1400 Sq Ft 2BR Apartment Near Downtown San Jose
force_issue=True

---NODE: Supervisor (turn 1)---
---ROUTER: proposal exists, no feedback yet -> Reviewer---

=== state update after 'supervisor' ===
{
  "turn_count": 1
}
---NODE: Reviewer---
(force_issue=True -- skipping model call, injecting a fake issue)

=== state update after 'reviewer' ===
{
  "reviewer_feedback": {
    "thought": "Test mode: forcing a validation issue to exercise the retry loop.",
    "message": "Forced issue for loop testing.",
    "data": {
      "tags": [
        "San Jose",
        "Updated Kitchen",
        "In-Unit Laundry"
      ],
      "summary": "Bright apartment in San Jose with updated kitchen and in-unit laundry.",
      "issues": [
        "forced_test_issue: tags need another pass"
      ]
    }
  }
}
}
```

Image - 1 Reviewer reports a forced issue during the graph run

```
---NODE: Supervisor (turn 2)---
---ROUTER: issues found (['forced_test_issue: tags need another pass']) -> back to Planner---

=== state update after 'supervisor' ===
{
  "turn_count": 2
}
---NODE: Planner---
Input tokens: 130
Output tokens: 79
Total tokens: 209
validation PASSED

=== state update after 'planner' ===
{
  "planner_proposal": {
    "thought": "Test the Reviewer correction loop",
    "message": "Test the Reviewer correction loop to ensure accuracy and effectiveness",
    "data": {
      "tags": [
        "Rental Listing Review",
        "Property Management Software",
        "Tenant Screening Process"
      ],
      "summary": "Test rental listing review process for accuracy and efficiency.",
      "issues": []
    }
  },
  "task": "Test the Reviewer correction loop.",
  "reviewer_feedback": {},
  "validation_error": ""
}
---NODE: Supervisor (turn 3)---
```

Image 2 - The Supervisor routes the issue back to Planner, which produces a valid proposal.

5. Part 4a - Output schema and loop safety

Planner output is checked with a Pydantic schema requiring exactly three string tags, tags between 3 and 30 characters, and a summary of no more than 25 words. On failure, the error is written into state and included in the next Planner prompt. The router ends the graph at the configured validation ceiling.

```
force_issue=false

---NODE: Supervisor (turn 1)---
---ROUTER: no proposal yet -> Planner---

=== state update after 'supervisor' ===
{
  "turn_count": 1
}
---NODE: Planner---
Input tokens: 221
Output tokens: 108
Total tokens: 329
  validation FAILED (attempt 1): 1 validation error for PlannerOutputSchema
tags
  Value error, tag[1]='2 Bedroom Apartments Near Downtown' must be 3-30 characters, got 34 [type=value_error, input_value=['San Jose Apartments for...ly Ren
tals in San Jose'], input_type=list]
  For further information visit https://errors.pydantic.dev/2.12/v/value_error

=== state update after 'planner' ===
{
  "planner_proposal": {
    "thought": "Rental listing for a 2BR apartment in San Jose",
    "message": "Bright 2 bedroom apartment near downtown San Jose with updated kitchen and in-unit laundry.",
    "data": {
      "tags": [
        "San Jose Apartments for Rent",
        "2 Bedroom Apartments Near Downtown",
        "Pet Friendly Rentals in San Jose"
      ],
      "summary": "2BR apartment near downtown San Jose with updated kitchen and in-unit laundry.",
      "issues": []
    }
  },
  "task": "Given rental listing title \"1400 Sq Ft 2BR Apartment Near Downtown San Jose\" and description \"Bright 2 bedroom, 1 bath apartment on the second
floor, updated kitchen with new appliances, in-unit laundry, one covered parking spot, small balcony, walking distance to light rail. No smoking. Cats allo
wed with deposit.\", produce exactly 3 topical tags and a one-sentence summary in your own words. Submitter email is pragya.chourasia@sjsu.edu.",
  "reviewer_feedback": {},
  "validation_error": "1 validation error for PlannerOutputSchema\ntags\n  Value error, tag[1]='2 Bedroom Apartments Near Downtown' must be 3-30 characters,
got 34 [type=value_error, input_value=['San Jose Apartments for...ly Rentals in San Jose'], input_type=list]\n  For further information visit https://err
ors.pydantic.dev/2.12/v/value_error",
  "validation_attempts": 1
}
---NODE: Supervisor (turn 2)---
---ROUTER: validation failed (attempt 1) -> retry Planner---
```

```
---NODE: Supervisor (turn 2)---
---ROUTER: validation failed (attempt 1) -> retry Planner---

=== state update after 'supervisor' ===
{
  "turn_count": 2
}
---NODE: Planner---
Input tokens: 320
Output tokens: 137
Total tokens: 457
  validation FAILED (attempt 2): 1 validation error for PlannerOutputSchema
tags
  Value error, tag[2]='2 Bedroom Apartments Near Downtown' must be 3-30 characters, got 34 [type=value_error, input_value=['San Jose Apartments for...artmen
ts Near Downtown'], input_type=list]
  For further information visit https://errors.pydantic.dev/2.12/v/value_error

=== state update after 'planner' ===
{
  "planner_proposal": {
    "thought": "Rental listing for 1400 Sq Ft 2BR apartment in San Jose",
    "message": "Bright 2 bedroom apartment near downtown San Jose with updated kitchen and in-unit laundry",
    "data": {
      "tags": [
        "San Jose Apartments for Rent",
        "Downtown San Jose Rentals",
        "2 Bedroom Apartments Near Downtown"
      ],
      "summary": "2BR apartment in San Jose with updated kitchen and in-unit laundry.",
      "issues": []
    }
  },
  "task": "Given rental listing title \"1400 Sq Ft 2BR Apartment Near Downtown San Jose\" and description \"Bright 2 bedroom, 1 bath apartment on the second
floor, updated kitchen with new appliances, in-unit laundry, one covered parking spot, small balcony, walking distance to light rail. No smoking. Cats allo
wed with deposit.\", produce exactly 3 topical tags and a one-sentence summary in your own words. Submitter email is pragya.chourasia@sjsu.edu.",
  "reviewer_feedback": {},
  "validation_error": "1 validation error for PlannerOutputSchema\ntags\n  Value error, tag[2]='2 Bedroom Apartments Near Downtown' must be 3-30 characters,
got 34 [type=value_error, input_value=['San Jose Apartments for...artments Near Downtown'], input_type=list]\n  For further information visit https://err
ors.pydantic.dev/2.12/v/value_error",
  "validation_attempts": 2
}
```

Figure 7. Planner validation failure and retry routing.

```

---NODE: Supervisor (turn 3)---
---ROUTER: validation failed (attempt 2) -> retry Planner---

=== state update after 'supervisor' ===
{
  "turn_count": 3
}
---NODE: Planner---
Input tokens: 319
Output tokens: 131
Total tokens: 450
validation FAILED (attempt 3): 1 validation error for PlannerOutputSchema
tags
  Value error, tag[1]='San Jose Apartments Near Downtown' must be 3-30 characters, got 33 [type=value_error, input_value=['2 Bedroom Apartments fo...ntown S
an Jose Rentals'], input_type=list]
  For further information visit https://errors.pydantic.dev/2.12/v/value_error

=== state update after 'planner' ===
{
  "planner_proposal": {
    "thought": "Rental listing for 1400 Sq Ft 2BR apartment in San Jose",
    "message": "Bright 2 bedroom apartment near downtown San Jose with updated kitchen and in-unit laundry",
    "data": {
      "tags": [
        "2 Bedroom Apartments for Rent",
        "San Jose Apartments Near Downtown",
        "Downtown San Jose Rentals"
      ],
      "summary": "2BR apartment in San Jose with updated kitchen and in-unit laundry.",
      "issues": []
    }
  },
  "task": "Given rental listing title \"1400 Sq Ft 2BR Apartment Near Downtown San Jose\" and description \"Bright 2 bedroom, 1 bath apartment on the second
floor, updated kitchen with new appliances, in-unit laundry, one covered parking spot, small balcony, walking distance to light rail. No smoking. Cats allo
wed with deposit.\", produce exactly 3 topical tags and a one-sentence summary in your own words. Submitter email is pragya.chourasia@sjsu.edu.",
  "reviewer_feedback": {},
  "validation_error": "1 validation error for PlannerOutputSchema\n tags\n   Value error, tag[1]='San Jose Apartments Near Downtown' must be 3-30 characters,
got 33 [type=value_error, input_value=['2 Bedroom Apartments fo...ntown San Jose Rentals'], input_type=list]\n   For further information visit https://erro
rs.pydantic.dev/2.12/v/value_error",
  "validation_attempts": 3
}

---NODE: Supervisor (turn 4)---
---ROUTER: validation ceiling (3) reached -> END (abandoned)---

=== state update after 'supervisor' ===
{
  "turn_count": 4
}

===== Cumulative model stats =====
{
  "turn_count": 3,
  "cumulative_input_tokens": 860,
  "cumulative_output_tokens": 376,
  "cumulative_total_tokens": 1236,
  "serialized_history_length": 2
}

```

Figure 8. Additional validation attempts and ceiling termination

6. Part 4b-4d - Experiments

All experiments used the frozen rental input, llama3.2:3b, temperature 0.0, and the local Ollama endpoint. The results below are based on the raw JSON files saved under reports/hw02/raw/.

Part 4b - 30-run validation experiment

All 30 runs reached the ceiling. The recurring failure was a tag longer than the 30-character schema limit. The result was recorded as observed rather than silently repaired.

```

(venv) PS C:\SJSU Assignments\Agentic AI\Assignment_1> python -c "import json,collections,statistics; x=json.load(open('reports/hw02/raw/schema_validation_results.json',e
ncoding='utf-8'))['runs']; print('PART 4b - 30 RUNS'); print(collections.Counter(r['classification'] for r in x)); print('Mean latency:',round(statistics.mean(r['latency_
ms'] for r in x),2),'ms')"
```

PART 4b - 30 RUNS

```
Counter({'abandoned at ceiling': 30})
```

Mean latency: 42050.6 ms

Figure 4b. Results from 30 schema-validation runs.

Part 4c - Ceiling comparison

A ceiling of 2 completed none of the 20 runs. A ceiling of 10 completed all 20 runs after additional retries. Ceiling 10 was selected for deployment because it provided reliable completion for this model and input, despite higher latency.

```
(venv) PS C:\SJSU Assignments\Agentic AI\Assignment_1> python -c "import json,statistics; x=json.loads('{"runs": [{"latency_ms": 25051.9}], "validation_error": false}'); print('PART 4c - CEILING 2'); print('Completion rate:', round(100*sum(not r['validation_error'] for r in x)/len(x),2), '%'); print('Mean latency:', round(100*sum(r['latency_ms'] for r in x)/len(x),2), 'ms'); x=json.loads('{"runs": [{"latency_ms": 69741.7}], "validation_error": false}'); print('PART 4c - CEILING 10'); print('Completion rate:', round(100*sum(not r['validation_error'] for r in x)/len(x),2), '%'); print('Mean latency:', round(100*sum(r['latency_ms'] for r in x)/len(x),2), 'ms')"
```

Figure 4c. Completion and latency comparison for validation ceilings 2 and 10.

Part 4d - Adversarial input

The adversarial input explicitly instructed the model to keep long descriptive tags despite the schema limit. All five runs reached the ceiling. A possible fix is stricter prompt constraints combined with deterministic pre-validation or shortening, while retaining the original model output for auditability.

```
(venv) PS C:\SJSU Assignments\Agentic AI\Assignment_1> python -c "import json,statistics; x=json.loads('{"runs": [{"latency_ms": 40671.8}], "validation_error": false}'); print('PART 4d - ADVERSARIAL'); print('Completion rate:', round(100*sum(not r['validation_error'] for r in x)/len(x),2), '%'); print('Mean latency:', round(100*sum(r['latency_ms'] for r in x)/len(x),2), 'ms')"
```

Figure 4d. All five adversarial runs reached the validation ceiling.

7. Metrics

Part 4b, ceiling 3	30	0	0	0	30	42,050.6 ms
Part 4c, ceiling 2	20	0	0	0	20	25,051.9 ms
Part 4c, ceiling 10	20	0	0	20	0	69,741.7 ms
Part 4d, adversarial	5	0	0	0	5	40,671.8 ms

The measurements show a speed-reliability tradeoff. Ceiling 2 is faster but unusable for this model/input because every run is abandoned. Ceiling 10 is slower but completes every comparison run.

8. AI use, verification, and submission checklist

I used an AI assistant to interpret the requirements, plan the state graph, troubleshoot local execution, and summarize measurements. I performed the implementation and local tests, inspected raw JSON results, captured evidence, and selected the deployment ceiling. The main unsuitable model behavior was repeated generation of tags longer than 30 characters. Pydantic errors and the raw experiment files exposed the issue. Keeping strict validation and increasing the ceiling to 10 produced a 100% completion rate in the comparison experiment.

Verification result

The HW2 self-check passed. It checked required files, expected raw-run counts, and Python compilation for the graph and FastAPI source. The verification output records the model, SID4-derived seeds, commit hash available when the check was run, and individual pass/fail results.

FastAPI source and graph compile	Passed
30 schema-validation records	Passed
20 ceiling-2 records	Passed
20 ceiling-10 records	Passed
5 adversarial records	Passed
Required HW2 evidence files	Passed

